

10. e. Multi – level Linked List
–Modified Insert After Element
–Time Complexity Analysis
–Worst Case[Balanced Tree]

[For Balanced Tree]

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

$$2T\left(\frac{N-1}{2}\right) + C \text{ can be approximated as } \approx 2T\left(\frac{N}{2}\right) + C$$

Method 1: The Simplified Master Theorem

This method relies on matching the equation to the specific polynomial form:

$$T(n) = aT\left(\frac{n}{b}\right) + cn^k$$

Step1 : Extract the constants:

$a = 2$ (The number of subproblems)

$b = 2$ (The division factor)

$c = C$ (The constant work done)

$k = 0$ (Because the constant work C doesnot scale with N , it can be written as $C \cdot N^0$)

$$2T\left(\frac{N}{2}\right) + C \cdot N^0$$

Step 2: Evaluate the Condition

The Simplified Master Theorem ask us to compare the constants ' a ' and ' b^k ' to determine which case applies.

- $a = 2$
- $b^k = 2^0 = 1$

Because $2 > 1$, we clearly see that $a > b^k$.

Step 3: Apply the correct case

- Case 1 requires $a < b^k$.
- Case 2 requires $a = b^k$.
- Case 3 requires $a > b^k$.

Our equation triggers Case 3 , which states that the time complexity is:

$$\Theta(n^{\log_b a})$$

Step 4: The Final Calculation

Substituting the values of a and b into the Case 3 solution:

$$\begin{aligned}\Theta(n^{\log_2 2}) \\ = \Theta(n^1)\end{aligned}$$

Final Answer: $\Theta(N)$

Method 2: Continuous Generalized Master Theorem

$$\begin{aligned}T(n) &= aT\left(\frac{n}{b}\right) + c \times f(n) \\ \text{or, } 2T\left(\frac{N}{2}\right) + C \times N^0\end{aligned}$$

Step 1 : Extract the variables:

$$\begin{aligned}a &= 2 \\ b &= 2 \\ f(N) &= N^0\end{aligned}$$

Step 2 Calculate $n^{\log_b a}$:

$$\therefore N^{\log_b a} = N^{\log_2 2} = N^1 = N$$

Step 3: Determine ε :

- Exponent of $N^{\log_b a}$ is 1.
- Exponent of $f(n) = N^0 = 0$.
- Difference (ε) is : $1 - 0 = 1$.
- Therefore $\varepsilon = 1$.

Step 4: Determine Cases:

1. If there exists a constant $\varepsilon > 0$ such that

$$f(n) = O(n^{\log_b a - \varepsilon}) \Rightarrow n^{\log_b a - \varepsilon} < n^{\log_b a} \text{ then } T(n) = \Theta(n^{\log_b a}).$$

2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \log^k n)$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

3. If there exists a constant $\varepsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \varepsilon})$, and if $f(n)$ additionally satisfies the regularity condition $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$.

We got $\varepsilon > 0$ i.e. $\varepsilon = 1$, for Case 2 it will be required:

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

even if $k = 0$, $f(n) = \Theta(n^{\log_2 2} \log^0 n) \Rightarrow \Theta(n^{\log_2 2} \times (\log n)^0) = \Theta(n^{\log_2 2} \times 1)$
 $= \Theta(n) [\log_a a = 1]$

But our $f(n) = C \times N^0$ or, $\Theta(1)$.

$$f(n) = \Theta(n) \neq f(n) = \Theta(1)$$

not satisfied.

We got $\varepsilon > 0$ i.e. $\varepsilon = 1$, for Case 3 it will be required:

$$f(n) = \Omega(n^{\log_b a + \varepsilon}) = \Omega(n^{\log_2 2 + 1}) = \Omega(n^{\log_2 2 + 1}) = \Omega(n^{1+1})$$

$$= \Omega(n^{1+1}) = \Omega(n^2)$$

$$i.e. F(n) = \Omega(n^{1+1})$$

$$\Omega(n^{1+\varepsilon}) \neq F(n) = \Theta(1)$$

Hence Case 3, also not possible.

As $\varepsilon > 0$, we proceed with case 1:

$$f(n) = O(n^{\log_b a - \varepsilon}) = O\left(\frac{n^{\log_b a}}{n^\varepsilon}\right) = O\left(\frac{n^{\log_b a}}{n^\varepsilon}\right) = O\left(\frac{n^{\log_2 2}}{n^1}\right)$$

$$= O\left(\frac{n^{\log_2 2}}{n^1}\right) = O\left(\frac{n^1}{n^1}\right) = O(n^{1-1}) = O(n^0) = O(1).$$

$$\text{As, } n^0 \text{ or } n^{1-\varepsilon} < n^1 \Rightarrow n^{\log_b a - \varepsilon} < n^{\log_b a}$$

Therefore, it satisfies Case 1,

$$\text{Hence } T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n^1) = \Theta(n)$$

[Balanced Structure] – By Substitution Method.

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

$$2T\left(\frac{N-1}{2}\right) + C, \text{ can also be written as : } 2T\left(\frac{N - (2-1)}{2}\right) + C$$

$$\begin{aligned} \text{Now, } T\left(\frac{N-1}{2}\right) &= 2T\left(\frac{\frac{N-1}{2} - 1}{\frac{2}{1}}\right) + C = 2T\left(\frac{\frac{N-1-2}{2}}{\frac{2}{1}}\right) + C \\ &= 2T\left(\frac{N-3}{2} \times \frac{1}{2}\right) + C \\ &= 2T\left(\frac{N-3}{4}\right) + C \end{aligned}$$

$$\text{Substituting it in : } 2T\left(\frac{N-1}{2}\right) + C$$

$$\Rightarrow 2\left[2T\left(\frac{N-3}{4}\right) + C\right] + C$$

$$\Rightarrow 2^2 T\left(\frac{N-3}{2^2}\right) + 2C + C$$

$$\Rightarrow 2^2 T\left(\frac{N - (2^2 - 1)}{2^2}\right) + 2C + C$$

$$\begin{aligned} \text{Now, } T\left(\frac{N-3}{4}\right) &= 2T\left(\frac{\frac{N-3}{4}-1}{2}\right) + C = 2T\left(\frac{\frac{N-4-3}{4}}{2}\right) + C \\ &= 2T\left(\frac{N-7}{8}\right) + C \end{aligned}$$

$$\text{Substituting it in : } 2^2 T\left(\frac{N-3}{4}\right) + 2C + C$$

$$\Rightarrow 2^2 \left[2T\left(\frac{N-7}{8}\right) + C \right] + 2C + C$$

$$\Rightarrow 2^3 T\left(\frac{N-7}{2^3}\right) + 2^2 C + 2C + C$$

$$\Rightarrow 2^3 T\left(\frac{N-(2^3-1)}{2^3}\right) + 2^2 C + 2C + C$$

Thus ,if it runs upto `k` times we get:

$$\Rightarrow 2^k T\left(\frac{N-(2^k-1)}{2^k}\right) + 2^{k-1}C + \dots + 2^2 C + 2C + C$$

$$\Rightarrow 2^k T\left(\frac{N-(2^k-1)}{2^k}\right) + C (2^{k-1} + \dots + 2^2 + 2^1 + 2^0)$$

Now ,here $2^{k-1} + \dots + 2^2 + 2^1 + 2^0$ is geometric progression.

Where number of terms counting from $2^0, 2^1, \dots, 2^{k-1} = k$ terms

We can also find it out by:

Number of integers from a to $b = b - a + 1$.

Where b is ending index.

a is starting index.

Here, $b = k - 1$ and $a = 0$.

$(k - 1) - 0 + 1 = k$ terms.

By formula : $S_n = a \left(\frac{r^n - 1}{r - 1} \right)$, where $a = \text{starting value} : 2^0 = 1$

$r = 2$

$n = \text{number of terms} = k$

$$S_n = 2^0 \left(\frac{2^k - 1}{2 - 1} \right) = 1 \left(\frac{2^k - 1}{1} \right) = 2^k - 1.$$

$$\therefore 2^k T \left(\frac{N - (2^k - 1)}{2^k} \right) + C(2^k - 1)$$

As, we know, $T(0) = 1$, i. e. base case at 0 then:

$$\Rightarrow \frac{N - (2^k - 1)}{2^k} = 0$$

$$\Rightarrow N - (2^k - 1) = 0$$

$$\Rightarrow N - 2^k + 1 = 0$$

$$\Rightarrow N + 1 = 2^k$$

Applying, $2^k = N + 1$ in formula : $2^k T \left(\frac{N - (2^k - 1)}{2^k} \right) + C(2^k - 1)$,

We get:

$$\Rightarrow (N + 1) \times T \left(\frac{N - ((N + 1) - 1)}{N + 1} \right) + C((N + 1) - 1)$$

$$\Rightarrow (N + 1) \times T \left(\frac{N - N}{N + 1} \right) + C(N)$$

$$\Rightarrow (N + 1) \times T \left(\frac{0}{N + 1} \right) + C(N)$$

$$\Rightarrow (N + 1) \times T(0) + C(N) \left[\begin{array}{l} \text{since, } \frac{0}{a} = 0, \text{ where } a > 0 \\ \text{As } N + 1, \therefore N \geq 0. \end{array} \right]$$

As, $T(0)$ is constant, let $T(0) = c_0$, knowing $T(0) = 1$, and we can implement 1 instead of $T(0)$ and end the process and we

can also replace `C` with 1 (as it is a standard) but lets keep it c_0 and continue:

$$\Rightarrow (N + 1) \times c_0 + c \times N [C \text{ or } c \text{ represents same constants}]$$

By applying Big – O , removing all constants we get:

$$\Rightarrow O((N + 1)) + O(N)$$

$$\Rightarrow O(N)$$

Balanced Structure [Recurrence Tree]

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

Program:

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}

void insertAfterElement()
{
    if (head == nullptr)
    {
        cout << "List is empty. Please insert an element first." << endl;
    }
}
```

```

        return;
    }

    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    Node *temp = searchNode(head, target);

    if (temp == nullptr)
    {
        cout << "Element " << target << " not found anywhere
in the list." << endl;
        return;
    }

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else

```



```

    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}
int main()
{
    int choice;
    while (true)
    {
        cout << "4. Insert After Element (Link or Child)" <<
endl;
        switch (choice)
        {
            case 4:
                insertAfterElement();
            default:
                cout << "Invalid choice" << endl;
        }
    }
    return 0;
}

```

Work Per Problem or Cost Per Problem :

Here is exactly what the CPU is spending cycles doing when holding a single node in its hand during searchNode:

- 1. Evaluate an if statement: if (current == nullptr) – checking if a pointer is null.***
- 2. Evaluate another if statement: if (current-> data == target) – integer comparison.***
- 3. Evaluate a third if statement: if (foundInChild != nullptr) – checking the result of the first recursive branch.***
- 4. Return a memory address: Handing a pointer back to the previous stack frame.***

That physical CPU effort is the Work per Problem or Cost Per Problem.

And Cost Per Problem is : C or 1 i.e. Constant.

Level (k)	No. of problems	Problem Size	Work /Cost Per Problem (Print and free memory)	Work done = Work per Problem $\times$ No. of Problems
0	1	n	C	$1 \times C = C$
1	2	$\frac{n-1}{2}$	C	$2 \times C = 2C$
2	4	$\frac{n-3}{4}$	C	$4 \times C = 4C$
3	8	$\frac{n-7}{8}$	C	$8 \times C = 8C$
.	.	.	.	.
k	2^k	$\frac{N - (2^k - 1)}{2^k}$	C	$2^k \times C = 2^k C$
(Base Case)	$N + 1$	(Base Case)	$T(0)$	$(N + 1) \times T(0)$

Q) Why ' $N + 1$ ' Base Case?

The Worst Case (Target NOT Found)

Base Case Hits: $N + 1$

If the target element does not exist anywhere in the linked list, searchNode has to prove it. To do that, it is forced to do a complete, exhaustive search of the entire structure.

In this scenario, searchNode behaves exactly like destroyNodeRecursively:

- *It visits all N real nodes.*
- *It follows all $2N$ pointers.*
- *It traverses the $N - 1$ valid connections.*
- *It hits all the leftovers: $N + 1$ nullptr pointers.*

That is :

Imagine of building a perfectly balanced multi – level linked list with exactly N real nodes.

1. *Total Pointers: Every single node in your structure contains exactly two pointers (child and next). Therefore, in a list of N nodes, there are exactly $2N$ total pointers physically sitting in memory.*
2. *Connecting the Nodes: To connect N nodes together into a single tree, we have to use exactly $N - 1$ pointers.*
(Think about it: to connect 2 nodes, it need 1 pointer. To connect 3 nodes, need 2 pointers).
3. *The Leftovers: If we have $2N$ total pointers, and we used $N - 1$ of them to wire the structure together, how many pointers are left pointing to absolutely nothing?*

Let's do the math:

$$\begin{aligned} & 2N - (N - 1) \\ &= 2N - N + 1 \\ &= N + 1 \end{aligned}$$

- $N + 1$ (*Leftover Pointers*): If we subtract the pointers we used to build the tree from the total pointers we started with, we get the leftovers:

$$2N - (N - 1) = 2N - N + 1 = N + 1$$

Since those $N + 1$ pointers were never used to connect to another node, they are left empty. In C++, an empty pointer is a `nullptr`.

Derivation of `k` :

Now we have to derive level `k` first:

$$\text{Problem size} = \frac{N - (2^k - 1)}{2^k}$$

And we know, $T(0)$ is our base case .

The tree stops growing (it hits the base case) when the problem size reaches 0.

So, to find the final level k , we set our pattern equal to 0 and solve for k :

$$\frac{N - (2^k - 1)}{2^k} = 0$$

Multiplying 2^k on both the sides:

$$\Rightarrow N - (2^k - 1) = 0$$

Moving the $2^k - 1$ to the other side, we get:

$$\Rightarrow N = 2^k - 1$$

Add 1 to both sides to isolate the exponential term:

$$\Rightarrow N + 1 = 2^k$$

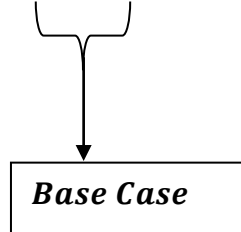
Take the base – 2 logarithm of both sides.

Because the base of our exponent is 2 (from 2^k), we use the base – 2 logarithm ($\log_2$) to target it.

$$\begin{aligned}\Rightarrow \log_2(N + 1) &= \log_2 2^k \\ \Rightarrow \log_2(N + 1) &= k \times \log_2 2 \\ \Rightarrow \log_2(N + 1) &= k [\log_a a = 1]\end{aligned}$$

$$\therefore k = \log_2(N + 1)$$

We got, $N + 1 = 2^k$ and $k = \log_2(N + 1)$



Hence full table is:

Level (k)	No. of problems	Problem Size	Work /Cost Per Problem (Print and free memory)	Work done = Work per Problem × No. of Problems
0	1	n	C	1 × C = C
1	2	$\frac{n-1}{2}$	C	2 × C = 2C
2	4	$\frac{n-3}{4}$	C	4 × C = 4C
3	8	$\frac{n-7}{8}$	C	8 × C = 8C
⋮	⋮	⋮	⋮	⋮
k	2^k	$\frac{N - (2^k - 1)}{2^k}$	C	2^k × C = 2^kC
⋮	⋮	⋮	⋮	⋮
log₂(N + 1) (Base Case)	N + 1	$\frac{N - (2^{\log_2(N+1)} - 1)}{2^{\log_2(N+1)}}$ = 0 (Base Case)	T(0)	(N + 1) × T(0)

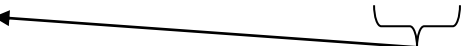
$$\frac{N - (2^{\log_2(N+1)} - 1)}{2^{\log_2(N+1)}} \text{ (Base Case)}$$

$$= \frac{N - (N + 1 - 1)}{N + 1}$$

$$= \frac{N - N - 1 + 1}{N + 1}$$

$$= \frac{0}{N+1} = 0$$

Also ,no. of problems : 2^k , when $k = \log_2(N+1) \Rightarrow 2^{\log_2(N+1)} \Rightarrow N+1$

We can also take from base level : $\log_2(N+1)$, Number of Problem is : $N+1$. 

$$\text{As, } 2^k = N+1.$$

**[From Mathematical Terms apart from
Pointer terms explanation.]**

Now lets calculate:

Now , we can see growth : $C + 2C + 4C + \dots + 2^{k-1}C + (N+1) \times T(0)$.

Note , $2^k = N+1$ (Already proved above)

$\therefore$ The series is actually,

$$\Rightarrow C + 2C + 4C + \dots + 2^{k-1}C + 2^k \times T(0) \text{ [Where } 2^k = N+1]$$

**[We know $T(0)$ is Constant
But already we know its value.]**

$$\text{Hence , } C + 2C + 4C + \dots + 2^{k-1}C + (N+1) \times T(0).$$

$$\Rightarrow C(1 + 2 + 4 + \dots + 2^{k-1}) + (N+1) \times T(0).$$

Applying geometric progression(series) :

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^n = a \left(\frac{r^{n+1} - 1}{r - 1} \right), \text{ where}$$

$S(n)$ is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

We get:

$$\Rightarrow C(1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + \dots + 1 \times 2^{k-1}) + (N + 1) \times T(0).$$

$$\Rightarrow C \left(1 \times \left(\frac{2^{k-1+1} - 1}{2 - 1} \right) \right) + (N + 1) \times T(0).$$

or we can directly apply geometric series , for $[a \times r^{n-1}]$

$$= a \left(\frac{r^n - 1}{r - 1} \right)$$

$$\Rightarrow C \left(1 \times \left(\frac{2^k - 1}{2 - 1} \right) \right) + (N + 1) \times T(0).$$

$$\Rightarrow C(2^k - 1) + (N + 1) \times T(0).$$

We know , $k = \log_2(N + 1)$, applying in the Equation we get:

$$\Rightarrow C(2^{\log_2(N+1)} - 1) + (N + 1) \times T(0).$$

$$\Rightarrow C((N + 1) - 1) + (N + 1) \times T(0). [Note, $a^{\log_a b} = b$]$$

$$\Rightarrow CN + (N + 1) \times T(0)$$

We know $T(0) = 1$, we can apply that here but instead we can write, $T(0)$ or t_0 as C_0 representing constant:

$$\Rightarrow CN + (N + 1) \times C_0$$

Applying Big O , in the equation removing all the constants:

$$\Rightarrow O(CN + (N + 1) \times C_0)$$

$$\Rightarrow O(N) + O(N + 1)$$

$$\Rightarrow O(N) + O(N + 1)$$

$$\Rightarrow O(N) + O(N)$$

$$\Rightarrow O(N + N)$$

$$\Rightarrow O(2N)$$

$$\Rightarrow O(N)$$

Alternatively,

Note:, if we take C and C_0 as 1 unit time, which will lead to the equation:

$$= N + N + 1$$

$$= 2N + 1$$

$$= O(N)[\text{Alternatively}]$$

[For Balanced Tree] – AkraBazzi Theorem

Multi – level Linked List

–Destroy List(Main List and Child)

–Time Complexity Calculation using AkraBazzi Method

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

Solution

$$a = 2 \text{ and } b = 2$$

$$\therefore \frac{a}{b^p} = 1, \text{ only when } 2 = 2^p \text{ or } \frac{2}{2^p} = 1, \text{ when } p = 1.$$

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{f(u)}{u^{p+1}} du\right)\right)$$

**Therefore, as per Akra – Bazzi theorem ,
the solution can be given as follows:**

$$2T\left(\frac{N-1}{2}\right) + C$$

$$u = n - 1 \quad , f(n) = C \text{ and } p = 1$$

If

$$f(x) = C,$$

then f is a constant function. Therefore,

$$\begin{aligned} f(n) &= C, \\ f(u) &= C, \\ f(n-1) &= C, \\ f\left(\frac{n}{2}\right) &= C. \end{aligned}$$

$$\therefore f(n) = c$$

$$\Theta\left(n^p\left(1 + \int_1^n \frac{f(n)}{n^{p+1}} dn\right)\right)$$

$$\Rightarrow \Theta\left(n^1\left(1 + \int_1^n \frac{c}{n^{1+1}} dn\right)\right)$$

$$\Rightarrow \Theta\left(n\left(1 + \int_1^n \frac{c}{n^2} dn\right)\right)$$

$$\Rightarrow \Theta\left(n\left(1 + c\left(\int_1^n \frac{1}{n^2}\right) dn\right)\right)$$

$$\Rightarrow \Theta \left(n \left(1 + c \left(\int_1^n n^{-2} \right) dn \right) \right)$$

Lets solve $\int_1^n n^{-2} dn$ this part first:

$$\text{We know: } \int x^a = \frac{x^{a+1}}{a+1}$$

$$\int_1^n n^{-2} \Rightarrow [n^{-2}]_1^n \Rightarrow \left[\frac{n^{-2+1}}{-2+1} \right]_1^n \Rightarrow \left[\frac{n^{-1}}{-1} \right]_1^n \Rightarrow \left[\frac{1}{-1} \times n^{-1} \right]_1^n$$

$$\Rightarrow [-1 \times n^{-1}]_1^n \Rightarrow \left[-1 \times \frac{1}{n} \right]_1^n \Rightarrow \left[-\frac{1}{n} \right]_1^n$$

Now compute boundaries of $-\frac{1}{n}$:

$$\int_b^a f(x) d(x) = [f(x)d(x)]_b^a = F(b) - F(a)$$

$$= \lim_{x \rightarrow b-} F(x) - \lim_{x \rightarrow a+} F(x)$$

Here, $x = n$, Therefore,

$$\lim_{n \rightarrow n-} F(x) = \lim_{n \rightarrow n-} \left(-\frac{1}{n} \right), \text{ Since } \lim_{x \rightarrow x} f(x) = f(x)$$

$$\therefore \lim_{n \rightarrow n-} \left(-\frac{1}{n} \right) = -\frac{1}{n}$$

$$\lim_{n \rightarrow 1+} \left(-\frac{1}{n} \right) = \text{plugging 1 in place of } n \text{ we get:}$$

$$= -\frac{1}{1} = -1$$

$$\therefore F(b) - F(a)$$

$$\Rightarrow -\frac{1}{n} - (-1)$$

$$\Rightarrow -\frac{1}{n} + 1$$

Applying it in : $\Theta\left(n\left(1 + c\left(\int_1^n n^{-2}\right) dn\right)\right)$, we get:

$$= \Theta\left(n\left(1 + c\left(-\frac{1}{n} + 1\right)\right)\right)$$

$$= \Theta\left(n\left(1 + \left(-\frac{c}{n} + c\right)\right)\right)$$

$$= \Theta\left(n\left(1 - \frac{c}{n} + c\right)\right)$$

$$= \Theta\left(n \times 1 - n \times \frac{c}{n} + nc\right)$$

$$= \Theta(n - c + nc)$$

$$= \Theta(n + nc - c)$$

$$= \Theta(n)$$

Hence, through Akrabazzi Method it is proved that :

Time Complexity : $\Theta(n)$
